

AURA

Adaptive Universal Robot Agent

A robot that continuously builds, tests, and improves its internal world model while solving unseen tasks.

Project Plan — What We Are Building and How

Embodied AI Research Platform • Simulation-first • Self-improving world model

1. What we are building

AURA is an embodied AI research platform: a simulated warehouse robot whose defining capability is not any single skill, but a **self-improvement loop**. The robot maintains an internal *world model* — a learned neural simulator that predicts what will happen next given its current view of the world and a candidate action. It uses this model to imagine futures and plan. Crucially, after every action it compares its prediction with reality, measures its own prediction error, stores its failures, discovers *categories* of situations where it is unreliable, and then deliberately collects new experience in exactly those situations to repair its model.

Most robot learning systems follow a fixed pipeline: observe → predict → act. AURA adds an outer loop: act → measure mistakes → remember failures → cluster them → generate targeted practice → retrain → repeat. The robot curates its own curriculum from its own weaknesses.

1.1 The research question

Can a world model automatically detect the types of situations where it is unreliable, and actively collect targeted experience to improve only those failure modes?

Everything else in the project — the simulator, the vision encoder, the transformer, the planner — is engineering in service of answering this one question with numbers and graphs. This is what turns AURA from a demo into a research contribution.

1.2 What AURA is not

- Not a chatbot — it has a body, sensors, and actuators in a physics simulator.
- Not a standard RL project — the policy is not learned by reward maximization alone; planning happens inside a learned model, and learning is driven by prediction error.
- Not a classic robotics project — the hardware/simulation is a means; the research object is the self-improving world model itself.

1.3 A concrete example

Task: "Bring me the bottle." The robot sees a bottle, a chair, and a table. Before moving, it imagines candidate plans inside its world model: walking straight leads to a predicted chair collision, so that plan is rejected; walking around the chair leads to a predicted successful grasp, so that plan is executed. Later, the robot predicts a door will be open, but in reality it is closed. Prediction error spikes. That surprise is stored in failure memory, the "door" failure cluster grows, and the system schedules extra exploration around doors. The next version of the world model is measurably better at doors. That loop — surprise to targeted practice to measurable improvement — is the research.

2. System architecture

The system is composed of seven modules connected in two loops: an inner control loop (perceive → imagine → plan → act) that runs every timestep, and an outer self-improvement loop (detect failure → cluster → generate tasks → retrain) that runs periodically.

Module	Role	How it works
Perception encoder (DINOv3, frozen)	Turns each RGB camera frame into a compact 768-dimensional latent vector.	Pretrained vision foundation model used as a fixed feature extractor. Never trained — this saves months of compute.
World model (Transformer)	The robot's imagination: predicts the next latent state given current latent + action.	Sequence model over (latent, action) pairs; trained with MSE + contrastive loss on future latents. Can roll forward many steps.
Memory (FAISS + replay buffer)	Stores all experience as searchable latent vectors; separately stores failures.	FAISS enables nearest-neighbour search, which powers failure clustering and retrieval of similar past situations.
Planner (MPC with CEM)	Chooses actions by imagining futures.	Cross-Entropy Method samples action sequences, rolls each out inside the world model, scores imagined outcomes, executes the best first action.
Simulator (Isaac Lab + PhysX)	The environment: a warehouse with shelves, boxes, humans, forklifts, dynamic obstacles.	GPU-accelerated physics; ground truth for what actually happens after each action.
Prediction-error module	Measures surprise: distance between the imagined next latent and the real one.	High error means the model misunderstood this situation; the transition is flagged and stored in failure memory.
Self-improvement loop	The research contribution: converts failures into targeted training.	Failure classifier → cluster similar failures → generate synthetic practice tasks per cluster → collect data → retrain → evaluate → repeat.

Data flow: RGB camera → DINOv3 → latent representation → (world model + memory) → MPC planner → candidate actions → simulator → new observation → prediction-error module → self-improvement loop → back to the world model.

3. Technology stack

Component	Choice	Why
Language	Python	Ecosystem standard for ML and robotics research.
Deep learning	PyTorch	Flexible research framework; native Isaac Lab support.
Vision encoder	DINOv3 (frozen)	State-of-the-art self-supervised features; zero training cost.
Simulator	Isaac Lab (NVIDIA)	GPU-parallel environments; realistic warehouse assets; PhysX physics.

Component	Choice	Why
World model	Transformer	Handles long context of (latent, action) sequences; scales well.
Planner	Cross-Entropy Method (CEM)	Simple, gradient-free, robust MPC; easy to debug.
Memory	FAISS + replay buffer	Fast similarity search over millions of latent vectors.
Logging	Weights & Biases	Experiment tracking, loss curves, video logging.
Tracking (optional)	MLflow	Model registry and reproducible runs.

Hardware requirement: a CUDA-capable NVIDIA GPU (RTX 3070/4070-class or better; more VRAM is better) for Isaac Lab and world-model training, or equivalent cloud GPU access. Verify this before Month 1 — it is the project's hardest practical constraint.

4. Environment and dataset

4.1 Why a warehouse

The warehouse is chosen deliberately over self-driving (too complex, too crowded) and over toy gridworlds (too trivial). A warehouse naturally contains all four capabilities the loop must demonstrate: navigation, manipulation, obstacle avoidance, and planning — plus dynamic elements (moving humans, forklifts) that guarantee the world model will make interesting mistakes worth learning from.

4.2 Self-generated dataset

No ImageNet, no COCO, no 10 TB downloads. The robot generates its own data: it performs random walks in the simulator while recording (RGB frame, action, next RGB frame) transitions. One day of parallel simulation yields on the order of 2 million transitions. This is not just a convenience — it is philosophically required: the entire thesis is that the robot generates and curates its own experience. Later cycles of the loop replace random walks with failure-targeted exploration, which is the point of the paper.

5. How we build it: five training stages

The build is staged so that every stage is independently testable before the next begins. Get an ugly end-to-end version of the whole loop working early with small models, then improve each piece — never spend six months on infrastructure with no closed loop.

Stage	What is built	How, concretely	Done when
1. Encoding	Frozen DINOv3 feature pipeline.	Pass every recorded frame through DINOv3; cache 768-d latents to disk; build the FAISS index over them.	Nearest-neighbour lookup of any frame returns visually similar frames.

Stage	What is built	How, concretely	Done when
2. World model	Transformer future-predictor.	Input: sequence of (latent, action). Output: next latent(s). Loss: MSE + contrastive term so distinct futures stay separable. Train on the 2M random-walk transitions.	1-step and 5-step prediction error on held-out data beats a copy-last-frame baseline by a clear margin.
3. Planner	CEM model-predictive control.	Sample N action sequences; roll each through the world model; score (goal progress, predicted collisions); refit sampling distribution; iterate; execute best first action; replan every step.	Robot navigates to goals and avoids static obstacles using imagination only.
4. Failure detection	Prediction-error module + failure memory.	After each real step, compute distance between predicted and actual latent; transitions above a calibrated threshold are written to failure memory with full context.	Surprising events (e.g. a human moves a box the model predicted still) are reliably flagged; error histograms look sane.
5. Self-improvement	The full outer loop — the paper.	Cluster failure latents (FAISS/k-means); label clusters (doors, moving humans, glass, chairs); auto-generate simulator scenarios per cluster; collect targeted data; retrain; re-evaluate per cluster.	Per-cluster prediction error and task success measurably improve cycle over cycle, faster than a uniform-retraining baseline.

The novelty lives in Stage 5. Prior work (e.g. Plan2Explore) explores toward generic model uncertainty; AURA's claim is stronger: failures are clustered into *semantic categories*, and synthetic practice tasks are generated *per category*. The paper must therefore compare against (a) uniform retraining on all failures and (b) uncertainty-bonus exploration, and show that targeted, clustered practice adapts faster.

6. Nine-month timeline

Month	Milestone	Deliverable
1	Learn Isaac Lab: spawn robot, attach camera, basic navigation.	Robot teleoperates in the warehouse; frames stream to disk.
2	Trajectory collection and dataset pipeline.	~2M (frame, action, next-frame) transitions stored and indexed.
3	DINOv3 encoder integration and latent generation.	Cached latent dataset + FAISS index.
4	Transformer world model training.	Held-out prediction curves in W&B; beats naive baselines.
5	CEM planner on top of the world model.	Video: robot planning around static obstacles in imagination.

Month	Milestone	Deliverable
6	Adaptive memory: failure store and retrieval.	Queryable failure database with context replay.
7	Failure detection and clustering.	Named failure clusters with per-cluster error stats.
8	Full self-improvement loop closed.	At least two complete improve cycles run end to end.
9	Evaluation, paper, demo video.	Benchmarks vs baselines; written paper; polished demo.

Risk notes: Months 1–2 (Isaac Lab learning curve) and Month 8 (loop stability) are the schedule risks. Mitigation: run a tiny end-to-end version of Stages 1–5 by Month 4 with small models, so Month 8 is scaling rather than first integration.

7. Evaluation: the numbers that become graphs

Metric	What it shows
Task success rate	Overall competence on navigation/fetch tasks, per self-improvement cycle.
Collision rate	Safety of the planner's imagined rollouts vs reality.
Planning time	Practicality of CEM at chosen horizon and sample counts.
Prediction error (global and per failure cluster)	The core scientific signal: does targeted practice shrink exactly the errors it targets?
Adaptation speed	How many environment steps are needed to fix a failure mode, vs baselines.
Generalization to new warehouses	Whether improvements transfer to unseen layouts.
Number of retraining cycles / improvement per cycle	Learning-curve of the outer loop itself — the headline figure of the paper.

8. Positioning, baselines and scope discipline

8.1 Literature and baselines (weeks 1–3, before any code)

- **DreamerV3** — world-model RL; the reference for learning behaviour in imagination.
- **TD-MPC2** — planning in latent space; the closest planning baseline to AURA's MPC.
- **Plan2Explore** — uncertainty-driven exploration; the baseline AURA must beat with clustered, targeted practice.
- **DINOv3** — the frozen vision backbone; read enough to justify freezing it.
- **Open X-Embodiment / NVIDIA GR00T** — context for where the field's large-scale efforts are; AURA deliberately does not compete here.

8.2 Scope discipline

The "universal" ambition (same brain for warehouse, house, office, hospital, drone, factory) is the long-term vision, not the 9-month deliverable — cross-embodiment generality is what large labs pursue with big teams. For this project, the credible generalization result is: prove the self-improvement loop convincingly in one warehouse environment, and demonstrate transfer to a second, unseen warehouse layout. "Universal AURA" is the future-work section.

8.3 Flagship presentation

AURA is intended as a flagship repository rather than one more disconnected project: a well-written paper, a professional demo video, benchmarks against the baselines above, reproducible experiments, clean modular open-source code, and documentation/website. One project at that level attracts more attention than ten small ones.

9. Immediate next actions

#	Action	Output
1	Confirm GPU access (local RTX-class or cloud) and install Isaac Lab.	"Hello warehouse" scene running.
2	2–3 week literature review of the five baselines above.	2-page related-work summary + chosen baselines.
3	Freeze the research question and the two comparison baselines in writing.	1-page research proposal.
4	Set up the repository skeleton: modules for encoder, world model, planner, memory, loop; W&B project.	Empty-but-runnable pipeline with tests.
5	Begin Month 1: spawn robot, camera, teleoperation, frame logging.	First recorded trajectories.

Summary: AURA is a warehouse robot built around one precise, testable idea — a world model that finds its own weaknesses, practices them on purpose, and provably gets better. The simulator, encoder, transformer and planner are the machinery; the self-improvement loop is the science.